

Scoping & Event Timeline

Detailed Procedure

How the v6 pipeline narrows from an engineer's prose description to a precise scope filter (Phase 1), then to a curated list of Top Event candidates ready for user confirmation (Phase 2). The two phases that decide whether FTA succeeds or wastes effort.

SCOPE	SKILLS INVOLVED	TABLES TOUCHED	COMPANION DOCS
Phase 1 + Phase 2	2 phase + 2 retrieval wrappers	UE_3gpp_signaling_log ONLY	fta-detailed- procedure · v6- design-review

CONTENTS

- 01 The pre-FTA prologue
- 02 Phase 1 · Principles
- 03 Phase 1 · Mechanics
- 04 Phase 1 · Worked example
- 05 Phase 2 · Principles
- 06 Phase 2 · Mechanics
- 07 Phase 2 · Building candidates
- 08 Phase 2 · Worked example
- 09 Handoff to Checkpoint A & Phase 3
- 10 Quick reference · halts & lookups

§01 The pre-FTA prologue

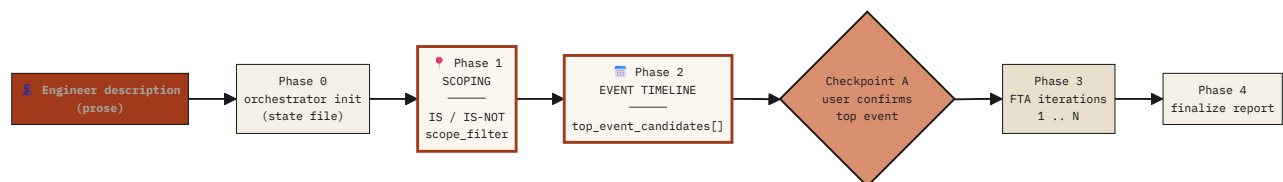
Phases 1 and 2 don't analyze causes. They prepare the ground for FTA by answering two prerequisite questions: *where in the system did this fail?* (Phase 1) and *what specifically did the failure look like in signaling?* (Phase 2). Both questions must be answered before the agent is allowed to hypothesize causes – and both must be answered without ever touching trace data or source code.

WHY THIS ORDERING MATTERS

If Phase 1's scope is wrong, FTA wastes effort building trees for procedures that aren't the real failure. If Phase 2's top event is wrong, FTA produces a confident-sounding root cause for the wrong symptom. The cost of getting these phases wrong is far higher than the cost of running them carefully – which is why v6 added Checkpoint A after Phase 2 (the user confirms the top event before FTA even starts).

FIG. 1 — WHERE PHASES 1 & 2 SIT IN THE PIPELINE

FIG-01



THREE PRINCIPLES THAT GOVERN BOTH PHASES

1. SEQUENTIAL NARROWING

Scope → symptom → cause. Phase 1 narrows from "5G HO fails" to a specific procedure on a specific RAT in a specific time window. Phase 2 narrows from "the procedure failed" to "the failure manifested as RRCReconfiguration with no following RAR." Phase 3 then asks "why?" – but only because the first two questions are answered.

2. TABLE ISOLATION (HARD)

Both phases may ONLY query `UE_3gpp_signaling_log.Trace data ( UE_Trace_log )` is forbidden until FTA Gate B. Code search is forbidden in both phases. The Python tool `log_query.py` enforces this at script entry with exit code 3 on violations – no skill discipline alone, an actual hard barrier.

3. SPEC-ANCHORED, NOT MEMORY-ANCHORED

Both phases derive "what should have happened" from `spec_query.py` `lightweight_procedure` , not from the agent's pre-trained memory. Every keyword used in any DB query traces to a spec query result in the same pipeline run. The expected message flow, the primary layers, the key timers – all come from GraphRAG output, never invention.

4. HONEST ABOUT UNCERTAINTY

If Phase 1 hits ambiguities, it HALTS and asks the engineer rather than guessing. If Phase 2's log coverage is sparse, it still produces candidates but marks confidence LOW and lets Checkpoint A surface the situation to the user. Neither phase fabricates evidence to look more confident.

SKILL / TOOL INVENTORY FOR PHASES 1 & 2

PHASE	OWNING SKILL	TOOLS ALLOWED	TOOLS FORBIDDEN
1	3gpp-scoping	<code>spec_query</code> <code>lightweight_procedure</code> (<code>need=is_is_not</code>) <code>log_query phase1</code> (<code>signaling, optional</code>)	<code>log_query</code> against <code>UE_Trace_log</code> <code>code_search</code> (any operation) <code>spec_query</code> other operations
2	3gpp-event-timeline	<code>spec_query</code> <code>lightweight_procedure</code> (<code>need=ecf</code>)	<code>log_query</code> against <code>UE_Trace_log</code>

PHASE	OWNING SKILL	TOOLS ALLOWED	TOOLS FORBIDDEN
		log_query phase2 (signaling, required)	code_search (any operation) spec_query other operations

§02 Phase 1 · Principles

WHAT PHASE 1 PRODUCES

The single output of Phase 1 is `phase1_scope_filter` – a structured object that downstream phases consult to know where to look. Every later phase reads from this object; nothing later in the pipeline can broaden the scope.

FIELD	TYPE	MEANING
procedure	string	3GPP procedure name (e.g. "Intra-AMF 5G Handover", "LTE Initial Attach")
rat	enum	LTE / 5G NR / NR-NSA / WCDMA
bands	string[]	Carrier bands involved (e.g. ["n78"], ["B7", "B41"])
layers	string[]	Protocol layers in scope (from spec lookup)
layers_excluded	string[]	Explicitly out of scope (the IS-NOT for layers)
condition	string	The state/trigger at failure (e.g. "Handover from Cell A to Cell B")
time_window	{start_ms, end_ms}	Window in which downstream phases query logs
discriminator	string	The biggest IS-vs-IS-NOT contrast (the most informative difference)
reproducibility	string	"always" / "intermittent ~X%"
ambiguities	string[]	MUST be empty before Phase 2. Otherwise HALT.
spec_lookup	object	Written by spec_query – primary_layers, key_timers, initiating_message, spec_refs

FIELD	TYPE	MEANING
signaling_sanity_check	object · optional	Written by log_query phase1 if invoked

WHY IS/IS-NOT AND NOT ANOTHER SCOPING TECHNIQUE

The IS/IS-NOT structure comes from the Kepner-Tregoe problem-solving method. It works for two specific reasons that matter to an LLM-driven pipeline:

1. **It forces the agent to articulate negatives.** "5G NR Band n78" is informative; "5G NR Band n78 but NOT LTE Band 7 in the same area" is much more informative. The negative half tells downstream phases what they can safely ignore.
2. **It yields a discriminator.** The biggest IS-vs-IS-NOT gap is the most likely place the failure lives. For "fails on 5G NR n78, succeeds on LTE B7 in same cell": the discriminator is "the 5G-specific layer or the n78-specific config." That's the place FTA should focus first.

The agent does not literally fill out a 4×2 grid in the output, but the reasoning behind the four dimensions (WHAT, WHERE, WHEN, EXTENT) drives `scope_filter.condition` , `discriminator` , `time_window` ,and `reproducibility` respectively. The final RCA report renders the grid for the engineer.

THE 4 IS/IS-NOT DIMENSIONS

	IS (THE FAILURE)	IS NOT (SIMILAR CONTEXTS THAT DON'T FAIL)
WHAT	The failing procedure – e.g. "5G NR Intra-AMF Handover"	Closely related procedures that succeed – e.g. "5G NR Initial Attach"
WHERE	Layers / bands / cells / PLMN where it happens	Layers / bands / cells where the same UE operates fine
WHEN	Conditions at failure – UE state, sequence position, time of day	Conditions when the procedure works normally
EXTENT	Severity, count, percentage – e.g. "100% of HO attempts, always"	The same failure being absent – e.g. "no failure on other procedures"

WHAT THE ENGINEER SEES VS WHAT THE SKILL DOES

The engineer types one prose paragraph. The skill silently builds the 4×2 grid as its internal reasoning, but only writes the structured fields to state. The grid appears in the final RCA report (Section 1 of the report template) so engineers downstream can verify the contrasts.

TOOL CALLS ALLOWED IN PHASE 1

- **Required:** `spec_query.py --operation lightweight_procedure --need is_is_not` – returns `primary_layers`, `key_timers`, `initiating_message`, `spec_refs`
- **Optional:** `log_query.py --phase-tag phase1 --table UE_3gpp_signaling_log` – confirms the procedure's initiating message actually appears in the capture (sanity check only)

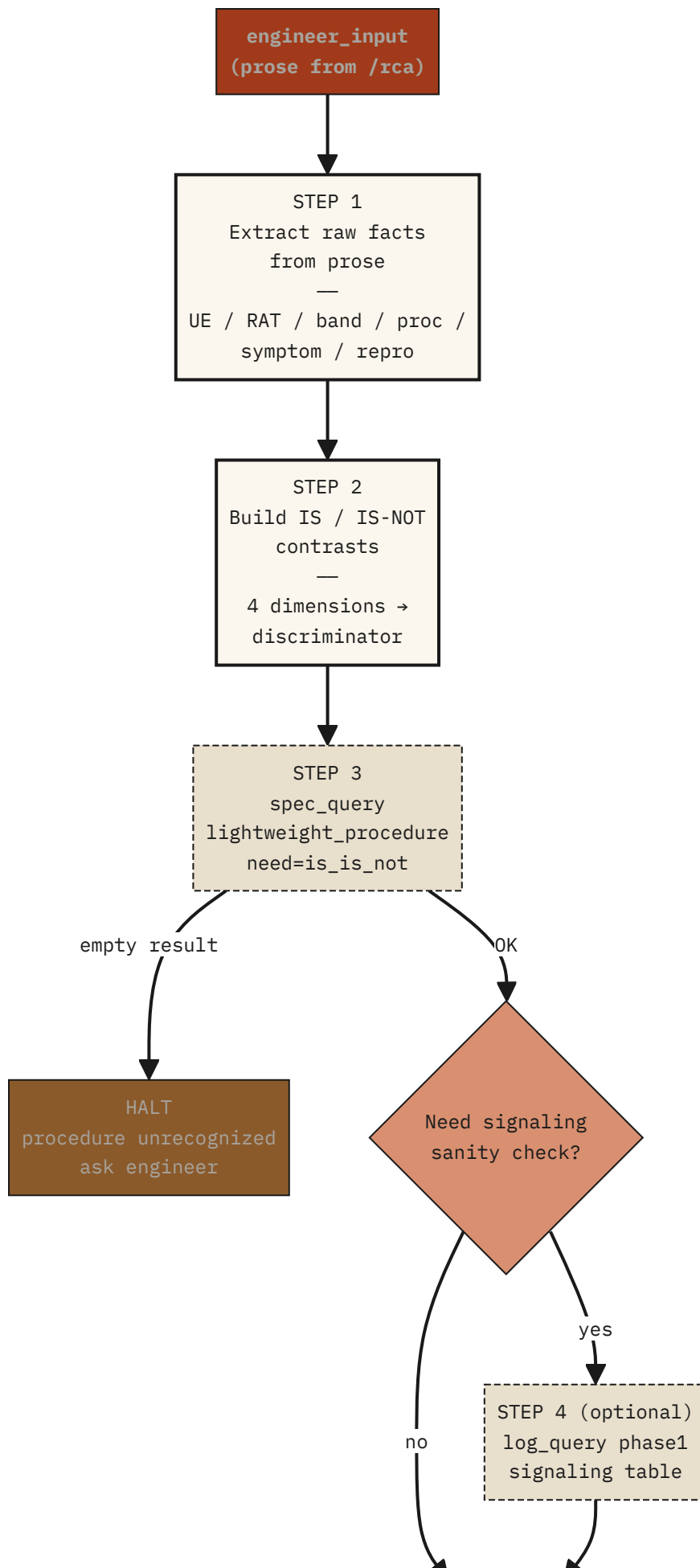
That's the entire tool surface of Phase 1. Two operations, both extremely lightweight. No code search, no trace queries, no other spec operations.

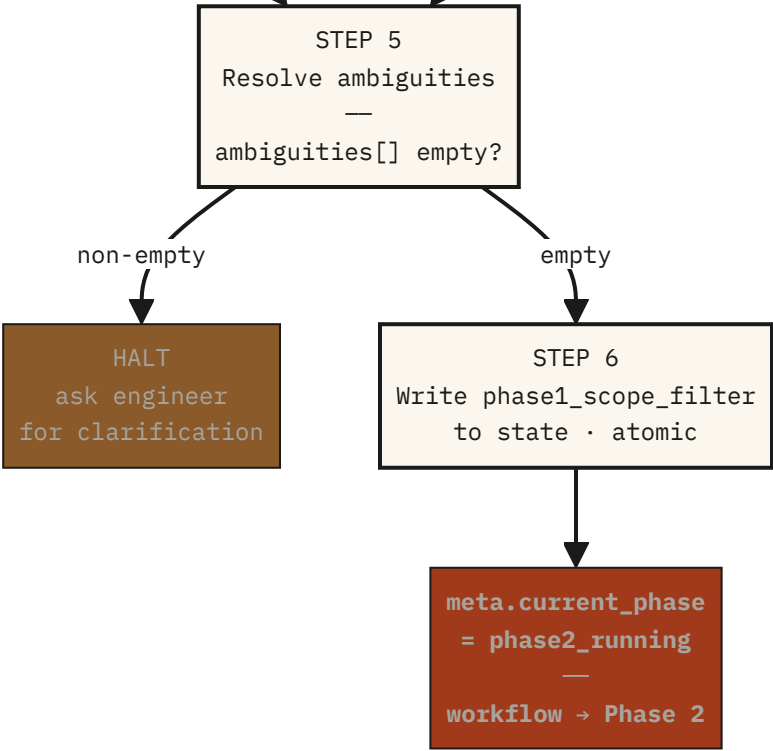
§03 Phase 1 • Mechanics

Six numbered steps. Steps 1–2 are pure reasoning over the engineer's prose. Step 3 is the one required tool call. Step 4 is the optional sanity check. Step 5 enforces no-ambiguity. Step 6 commits to state.

FIG. 2 — PHASE 1 PROCEDURE FLOW

FIG-02





STEP-BY-STEP MECHANICS

1

Extract raw facts from the engineer's prose

Read `meta.engineer_input` . Parse for the following – each becomes a candidate field or an ambiguity entry if absent:

- UE model / chipset / firmware version
- RAT (LTE / 5G NR / NR-NSA / WCDMA)
- Band(s), EARFCN/NR-ARFCN
- Network operator / PLMN
- Test scenario / procedure being performed
- Failure symptom as described by the engineer
- Conditions at failure (state, sequence, timing)
- Reproducibility (always / intermittent / specific trigger)
- Environment (lab / field / specific cell)

Items not mentioned in the prose are **not invented**. They go into `ambiguities[]` for the engineer to fill in if needed.

```
READmeta.engineer_input
```

```
NO TOOL CALLSpure parsing
```

2

Build IS / IS-NOT contrasts across 4 dimensions

For each of WHAT, WHERE, WHEN, EXTENT, articulate both columns. The IS-NOT column is generated by considering closely-related contexts where the same UE should work normally – same RAT but different procedure, same procedure but different band, same band but different cell, etc.

Pick the largest meaningful IS-vs-IS-NOT gap as

`scope_filter.discriminator` . This single field is the most important piece of Phase 1 output – it tells Phase 3 where to focus first.

```
WRITES (INTERMEDIATE)discriminator, condition, time_window, reproducibility
```

3

Call spec_query lightweight_procedure (required)

Invoke `spec_query.py --operation lightweight_procedure --need is_is_not` with the inferred procedure and RAT. The GraphRAG returns:

- `primary_layers` – typical protocol layers involved (e.g. ["NAS", "RRC", "MAC", "PHY"])
- `key_timers` – relevant procedure timers with default durations
- `initiating_message` – the spec-defined message that opens the procedure
- `spec_refs` – TS/clause references for the procedure

Copy `primary_layers` into `scope_filter.layers`. The full `spec_lookup` is written to `phase1_scope_filter.spec_lookup` by the Python tool automatically.

Halt condition: if the spec query returns empty (procedure unrecognized), append to ambiguities and HALT – ask the engineer to confirm the procedure name. This is rare in practice and almost always indicates a typo or non-standard procedure description.

```
TOOLSspec_query.py lightweight_procedure
EMPTY RESULTHALT with ambiguity note
```

4

(Optional) Signaling sanity check

Trigger only if at least one of these is true:

- Engineer description doesn't unambiguously identify the procedure (e.g. "handover failure" with no RAT specified)
- Time window is unclear and signaling can help bound it
- Confirming procedure presence in the capture would resolve scope ambiguity

Invoke `log_query.py --phase-tag phase1 --table UE_3gpp_signaling_log` with the initiating message as the keyword and a generous time window. The skill consults the result only to confirm the procedure appears in the capture – it does NOT analyze the procedure's outcome here (that's Phase 2's job).

Hard limit: Phase 1 must not query `UE_Trace_log`. The Python script enforces this with exit code 3.

TOOL (OPTIONAL) `log_query.py phase1`
TABLE RESTRICTION `UE_3gpp_signaling_log ONLY`

5

Resolve ambiguities

Review the `ambiguities[]` list accumulated so far. Each entry is a question the skill couldn't answer from the prose + tool results.

Hard rule: ambiguities MUST be empty before Phase 2 begins. If non-empty, HALT the workflow with a prompt asking the engineer to clarify each item. The state file is preserved so the engineer's clarification can be appended without restarting.

The rationale: ambiguities at Phase 1 compound at Phase 2 (the wrong messages get retrieved) and explode at Phase 3 (the wrong fault tree gets built). Halt-and-clarify is cheaper than running the whole pipeline on bad scope.

6

Atomic write to state file

Compose the complete `phase1_scope_filter` block including `completed_at` timestamp. Atomic write (`.tmp` → `mv`). Set `meta.current_phase = "phase2_running"` . The workflow's next dispatch will trigger Phase 2.

```
WRITESphase1_scope_filter (full block)
NEXT PHASEphase2_running → 3gpp-event-timeline
```

WHAT "AMBIGUITY RESOLUTION" LOOKS LIKE IN PRACTICE

Common Phase 1 ambiguities:

- RAT not mentioned ("UE handover fails" – LTE or 5G?)
- Procedure name vague ("attach fails" – Initial Attach? Combined Attach? Re-attach?)
- Time window absent ("yesterday at the office" – useless for log queries)
- Reproducibility unclear ("sometimes" – 10% or 90%?)

Each becomes one item in `ambiguities[]` . The HALT prompt lists them as a numbered list and asks the engineer to clarify before the pipeline proceeds.

§04 Phase 1 • Worked example

Using the same 5G handover failure example that runs through the FTA detailed procedure document.

PHASE 1 • ENGINEER INPUT (VERBATIM)

*UE 5G handover from Cell A to Cell B fails; HO Reconfig received but no service follows. UE: Snapdragon 888, FW 12.3. Network: Verizon 5G NR Band n78.
Repro: 100%, occurs at every handover from Cell A 1234 to Cell B 5678.
Capture window starts at 14:02:00, failure observed around 14:02:11.
LTE handovers in the same area work normally.*

STEP 1 – FACTS EXTRACTED

```
{
  ue: { model: "Snapdragon 888", firmware: "12.3" },
  rat: "5G NR",
  band: "n78",
  operator: "Verizon",
  procedure: "Intra-AMF Handover",          // inferred from "HO Reconfig"
  symptom: "HO Reconfig received but no service follows",
  reproducibility: "always (100%)",
  trigger: "Handover from Cell A 1234 to Cell B 5678",
  env: "field (cell IDs given)",
  failure_time: "~14:02:11"
}
```

STEP 2 – IS / IS-NOT CONTRASTS

	IS	IS NOT
WHAT	5G NR Intra-AMF Handover (Cell A → Cell B)	LTE handover (works fine in same area); 5G NR initial attach (no reports)
WHERE	5G NR Band n78, Cell A 1234 → Cell B 5678, Verizon PLMN	LTE handover in same area; other 5G NR cell pairs (no data either way)
WHEN	After RRCReconfiguration with reconfigurationWithSync, ~14:02:11	Other phases of the same call (call setup completes normally)
EXTENT	100% reproducibility; every HO attempt from Cell A to Cell B	Not observed on other procedures or on LTE

Discriminator (largest gap): "Fails on 5G NR Band n78 Cell A→Cell B; succeeds on LTE handovers in the same area." This points Phase 3 toward 5G-specific or n78-specific layers.

STEP 3 – SPEC_QUERY LIGHTWEIGHT_PROCEDURE (NEED=IS_IS_NOT)

```
spec_query.py --operation lightweight_procedure \  
  --procedure "Intra-AMF 5G Handover" \  
  --rat "5G NR" \  
  --need is_is_not \  
  --state-file /tmp/rca_state_20260520T140200Z.json  
  
→ {  
  "operation": "lightweight_procedure",  
  "need": "is_is_not",  
  "procedure": "Intra-AMF 5G Handover",  
  "primary_layers": ["RRC", "MAC", "PHY"],  
  "key_timers": [  
    { "timer": "T304", "duration_default": "Network-config; typical 1000ms",  
      "on_expiry": "Handover failure → reestablishment attempt" }  
  ],  
  "initiating_message": "RRCReconfiguration with reconfigurationWithSync",  
  "spec_refs": ["TS 38.331 §5.5.1", "TS 23.502 §4.9.1"]  
}
```

The skill copies `primary_layers` into `scope_filter.layers`. NAS is not in `primary_layers` (handover doesn't involve NAS layer once Reconfig is received), so NAS goes into `layers_excluded`.

STEP 4 – OPTIONAL SIGNALING SANITY CHECK

Because the engineer description is unambiguous (RAT, band, procedure all specified), the skill chooses to *skip* the optional sanity check. If the description had been "handover fails" with no RAT specified, the skill would have queried signaling for RRCReconfiguration occurrences to confirm 5G NR vs LTE.

STEP 5 – AMBIGUITIES CHECK

`ambiguities = []`. All fields populated from prose + spec_query. Proceed to Step 6.

STEP 6 – FINAL SCOPE_FILTER WRITTEN TO STATE

```
phase1_scope_filter = {
  completed_at: "2026-05-20T03:18:42Z",
  procedure: "Intra-AMF 5G Handover",
  rat: "5G NR",
  bands: ["n78"],
  layers: ["RRC", "MAC", "PHY"],
  layers_excluded: ["NAS"],
  condition: "Handover from Cell A 1234 to Cell B 5678, after RRCReconfiguration",
  time_window: { start_ms: 14020000, end_ms: 14023000 },
  discriminator: "Fails on 5G NR n78 Cell A→Cell B; succeeds on LTE handovers in same a",
  reproducibility: "always (100%)",
  ambiguities: [],
  spec_lookup: {
    primary_layers: ["RRC", "MAC", "PHY"],
    key_timers: [{ timer: "T304", duration_default: "~1000ms",
      on_expiry: "Handover failure → reestablishment" }],
    initiating_message: "RRCReconfiguration with reconfigurationWithSync",
    spec_refs: ["TS 38.331 §5.5.1", "TS 23.502 §4.9.1"]
  },
  signaling_sanity_check: null // not invoked in this run
}
```

Phase 1 complete. `meta.current_phase = "phase2_running"` .The workflow's next dispatch triggers `3gpp-event-timeline` .

§05 Phase 2 · Principles

WHAT PHASE 2 PRODUCES (AND WHAT IT DELIBERATELY DOES NOT)

Phase 2 produces the `phase2_ecf` block containing:

- `observable_symptoms` – the signaling timeline (classified events, missing events, log coverage assessment)
- `top_event_candidates[]` – **1 to 3** ranked candidate Top Events for the user to choose from
- `top_event = null` · *not set by this skill*
- `user_confirmation = null` · *not set by this skill*

THE V6 SPLIT: PHASE 2 PRODUCES; CHECKPOINT A DECIDES

v5 had Phase 2 set `top_event` directly and proceed. v6 splits this: Phase 2 produces a ranked list of candidates; the user chooses at Checkpoint A. The reason for the split is **snowball-error prevention**: if the agent silently picks the wrong top event, all of Phase 3 builds the wrong fault tree. The user gate intercepts this single most consequential decision in the pipeline.

THE FOUR EVENT CLASSIFICATIONS

Every signaling row retrieved in Phase 2 is classified into one of four buckets. The bucket determines whether the row becomes evidence, context, or a Top Event candidate.

✓ normal

Matches `expected_flow` ordering and timing. No failure indicators. Becomes timeline context only.

⚠ anomaly

First deviation from expected flow – wrong message, wrong timing, unexpected cause code. Marks where the procedure went off-script.

↪ consequence

✗ symptom

7/12/26, 2:52 PM	Phase 1 & Phase 2 — Detailed Procedure · 3GPP UE RCA v6
Follows from an earlier anomaly. Real but derivative – recovery attempts, retries, reestablishment.	Observable failure indicator at the application level – REJECT, timer expiry, no-service entry. Symptoms become top_event_candidates.

THE ANOMALY HEURISTICS (SIGNALING ONLY)

Anomalies at the signaling layer come from four families. PHY/MAC/RLC anomalies are NOT visible here – those surface in FTA Gate B, not Phase 2.

FAMILY	PATTERN IN SIGNALING	EXAMPLES
RRC	Timer expiry messages, unexpected state transitions	T304 expiry → RRC connection reestablishment attempt; unexpected RRCRelease mid-procedure
NAS	REJECT messages with non-zero cause code; unexpected EMM/5GMM state changes	ATTACH REJECT cause #11 (PLMN not allowed); SERVICE REJECT cause #7 (5GS services not allowed)
Message gaps	Expected message absent within the procedure's timer window	RRCReconfigurationComplete never sent after RRCReconfiguration received; ATTACH ACCEPT never received after ATTACH REQUEST
Out-of-order	REJECT before request; duplicate messages; ordering inversions	RRCConnectionReject before any RRCConnectionRequest; duplicate ATTACH REQUEST within seconds

THE THREE CONFIDENCE LEVELS FOR CANDIDATES

TIER 1 HIGH log_coverage is rich AND the candidate has explicit signaling evidence – REJECT with cause code, T-timer expiry message, or unambiguous symptom row.
TIER 2 MEDIUM

log_coverage is partial OR the candidate is inferred from message absence with strong context (e.g. no RAR within the expected window after a clean RACH preamble).

TIER 3

LOW

log_coverage is sparse OR the candidate is inferred from indirect signals. Still presented at Checkpoint A; the user decides whether to proceed, refine, or abort.

LOG COVERAGE ASSESSMENT

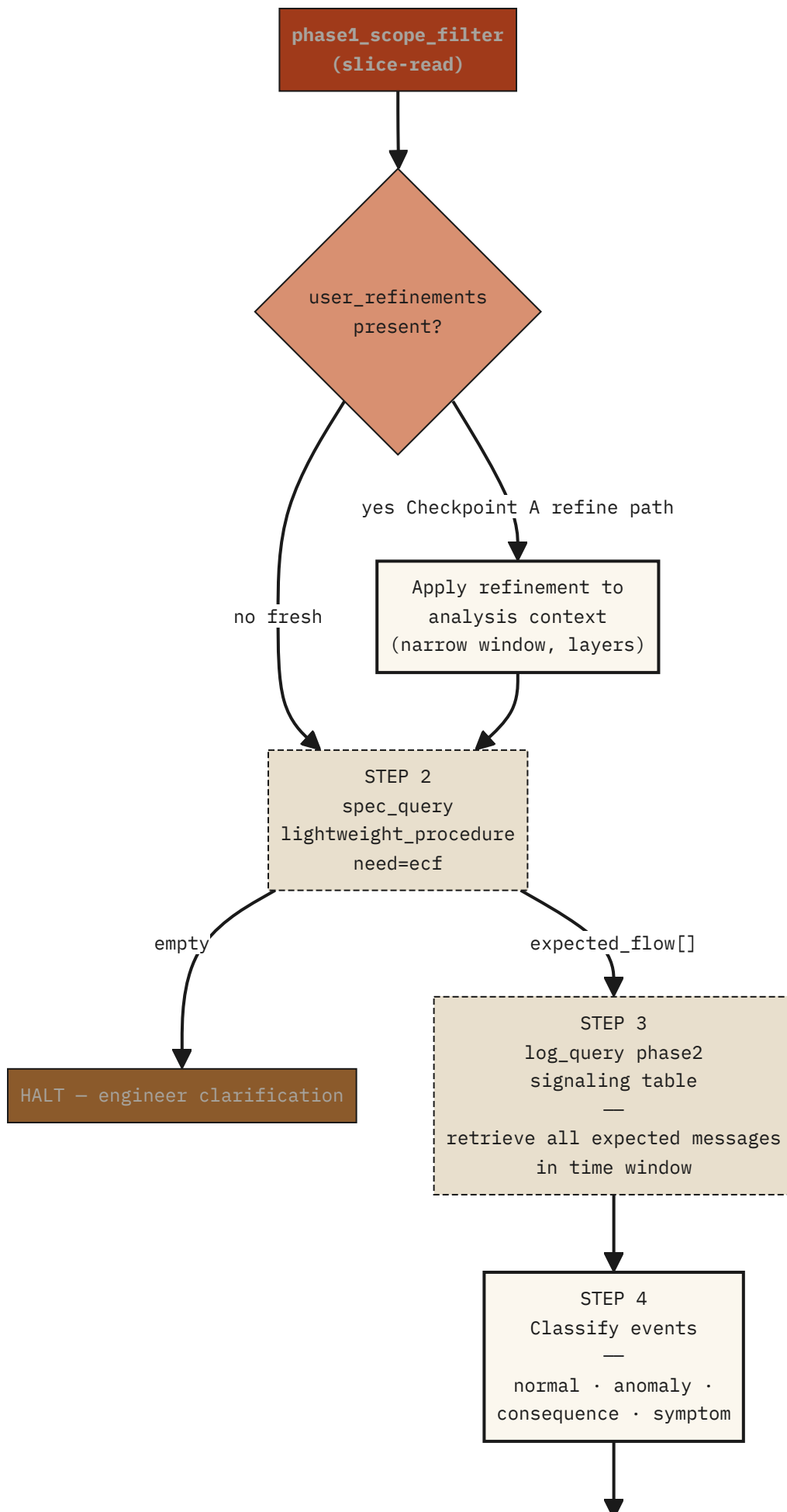
Phase 2 computes coverage as a ratio after retrieval. The ratio drives confidence assignment and the ambiguity handling path:

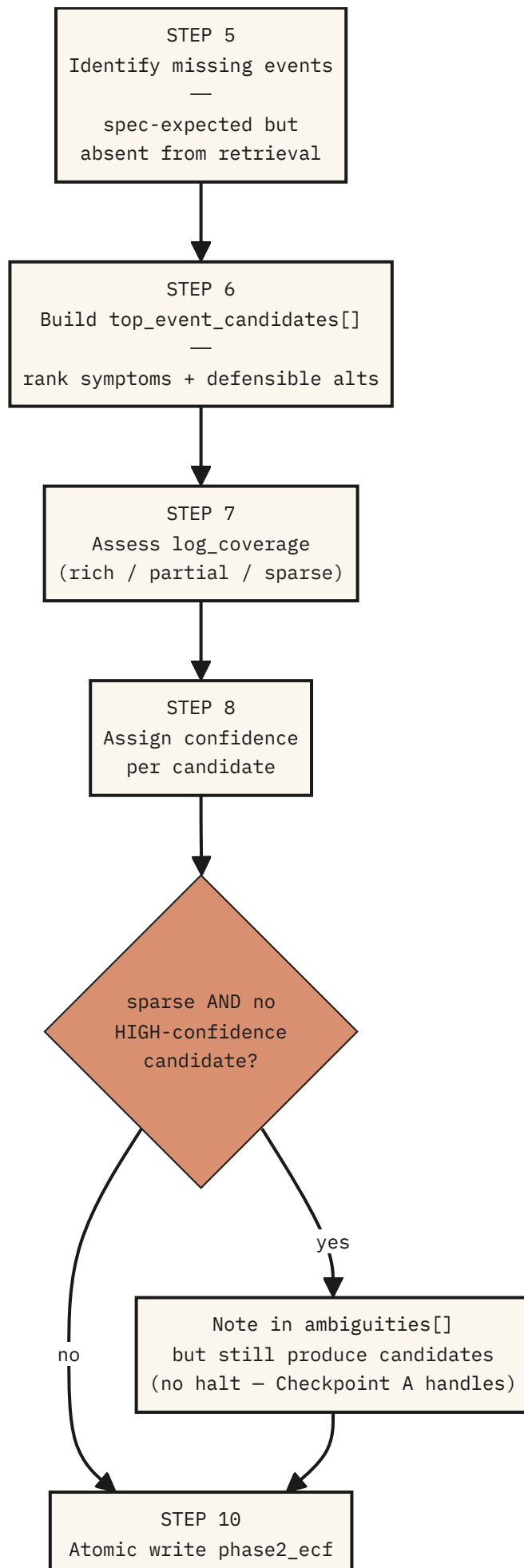
COVERAGE	RATIO	INTERPRETATION	EFFECT ON CONFIDENCE
RICH	≥ 0.8	Most expected messages present in signaling	HIGH possible
PARTIAL	0.4 – 0.79	Some expected messages missing – may be sampling or genuine gap	MEDIUM cap
SPARSE	< 0.4	Most expected messages missing – capture issue or wrong scope	LOW cap; suggest more capture in ambiguities

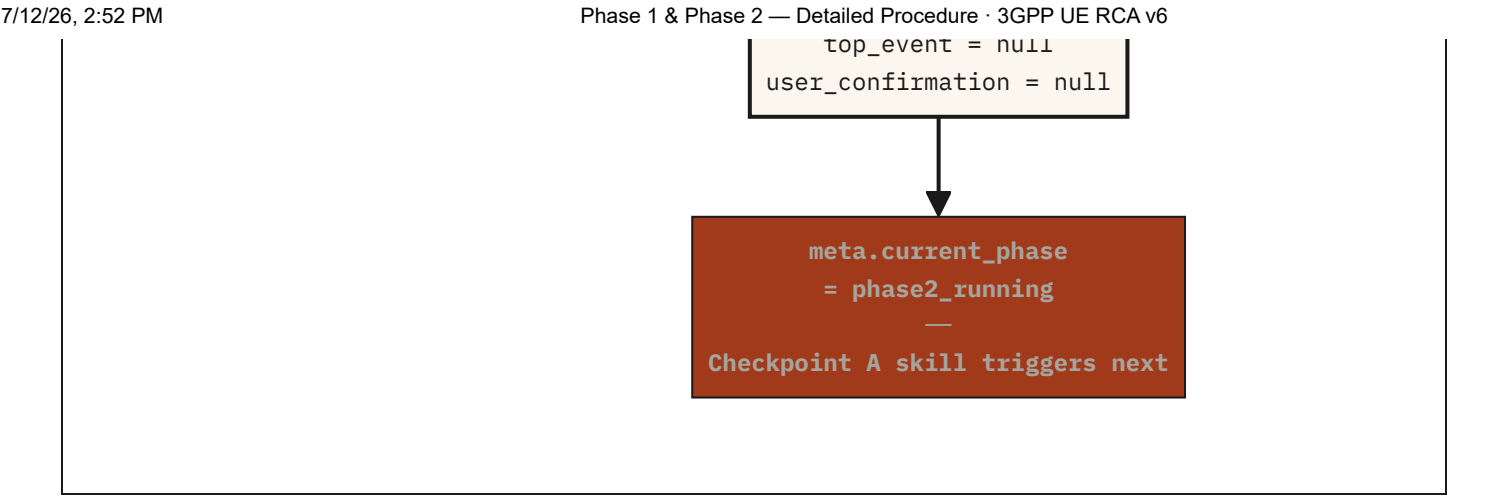
§06 Phase 2 · Mechanics

FIG. 3 — PHASE 2 PROCEDURE FLOW

FIG-03







STEP-BY-STEP MECHANICS

1	Read scope_filter (slice) and check for refinement context Slice-read phase1_scope_filter . Extract procedure, RAT, layers, time_window, condition, discriminator. If phase1_scope_filter.user_refinements is non-empty (re-run path from Checkpoint A "refine"), apply the refinement to the analysis context – typically narrowing the time window or adjusting the expected layers. This is the only Phase 2 path that uses user-supplied context beyond the initial scope. ----- READphase1_scope_filter (slice) REFINE PATHuser_refinements[] applied if present
---	--

2

Get expected message flow (spec_query)

Invoke `spec_query.py --operation lightweight_procedure --need ecf`. The GraphRAG returns:

- `expected_flow[]` – ordered messages with direction (UL/DL), layer, and sequence index
- `key_timers[]` – relevant procedure timers (already in `scope_filter.spec_lookup` but re-confirmed)
- `spec_refs[]` – citations for the expected flow

The `expected_flow` becomes the keyword list for Step 3 and the comparison baseline for Step 5.

Halt: if `expected_flow` is empty, the procedure isn't recognized by the GraphRAG – should have been caught in Phase 1, so this is a rare integrity error. Halt and ask engineer.

TOOLS `spec_query.py lightweight_procedure (need=ecf)`

OUTPUT `expected_flow_source` written to state

3

Retrieve signaling timeline (log_query)

Invoke `log_query.py --phase-tag phase2 --table UE_3gpp_signaling_log` with:

- `--keywords` = all message names from `expected_flow[]`
- `--time-window-*` = from `scope_filter.time_window`
- `--layers` = from `scope_filter.layers`

Returns up to 50 rows with timestamp, layer, direction, message_name, ie_values, cause_code. Plus a compressed key_events list (≤ 5) for in-context display.

Table isolation enforced by script. Phase 2 cannot touch UE_Trace_log; the script exits with code 3 if requested.

`TOOLlog_query.py phase2`

`RETURNS`rows[] + key_events[≤ 5]

4

Classify each retrieved event

Walk the retrieved rows in time order and assign one of four labels:

- **normal** — matches an entry in `expected_flow` at expected position with no failure indicators
- **anomaly** — first deviation from `expected_flow` (wrong message, wrong order, unexpected cause code)
- **consequence** — appears after an anomaly and is plausibly derivative (reestablishment attempt, retry)
- **symptom** — explicit failure indicator at application level: REJECT message, T-timer expiry, NoService entry. These are the candidates for `top_event`.

Capture `earliest_anomaly` = timestamp of the first anomaly-classified event, or null if no anomaly observed.

5

Identify missing events

For each entry in `expected_flow[]` whose message name does not appear in the retrieved rows, build a `missing_event` record:

```
{
  "expected_message": "RRCReconfigurationComplete",
  "expected_after": "RRCReconfiguration",
  "spec_ref": "TS 38.331 §5.5.1"
}
```

The `missing_events[]` list is consumed by Phase 3.2 to prioritize branch evaluation (branches whose `mandatory_messages` are in `missing_events` get evaluated first).

6

Build `top_event_candidates[]` – the v6 ranking algorithm

See §07 for the full ranking algorithm. In summary:

- **rank 1 (primary):** the LAST symptom in chronological order – the deepest unrecovered failure
- **rank 2-3 (alternatives):** earlier symptoms that pass the defensibility test

Never pad to 3 with hallucinated alternatives. List can be size 1, 2, or 3.

7

Assess `log_coverage`

Compute `coverage = retrieved_present_count / expected_flow_size` and bucket: rich (≥ 0.8), partial (0.4-0.79), sparse (< 0.4).

8

Assign confidence per candidate

For each candidate (see §05 for criteria):

- HIGH if coverage=rich AND explicit signaling evidence (REJECT/timer-expiry/symptom row exists for this candidate)
- MEDIUM if coverage=partial OR inferred-from-absence with strong context
- LOW if coverage=sparse OR indirect signals only

9

Ambiguity handling – DO NOT halt

If log_coverage=sparse AND no HIGH-confidence candidate: append a note to `phase1_scope_filter.ambiguities[]` ("Capture coverage sparse; consider extending capture window") but **do not halt the pipeline**.

Rationale: Checkpoint A is right after Phase 2. The user sees the sparse-coverage warning, sees the LOW-confidence candidates, and decides whether to `refine` (add more context), `reject` and `restart` (recapture and re-run from Phase 1), or proceed anyway. Halting Phase 2 would deny the user that choice.

10**Atomic write phase2_ecf – note top_event left null**

Write the full phase2_ecf block:

- completed_at set
- top_event = null – to be set by 3gpp-top-event-confirmation after Checkpoint A
- observable_symptoms fully populated (events, earliest_anomaly, missing_events, log_coverage, queried_tables, expected_flow_source)
- top_event_candidates[] populated with 1–3 ranked entries
- user_confirmation = null – to be set by the checkpoint skill

meta.current_phase remains at phase2_running ; the Checkpoint A skill will advance it to phase2_pending_confirmation .

WRITESphase2_ecf (top_event still null)

NEXT SKILL3gpp-top-event-confirmation (Checkpoint A)

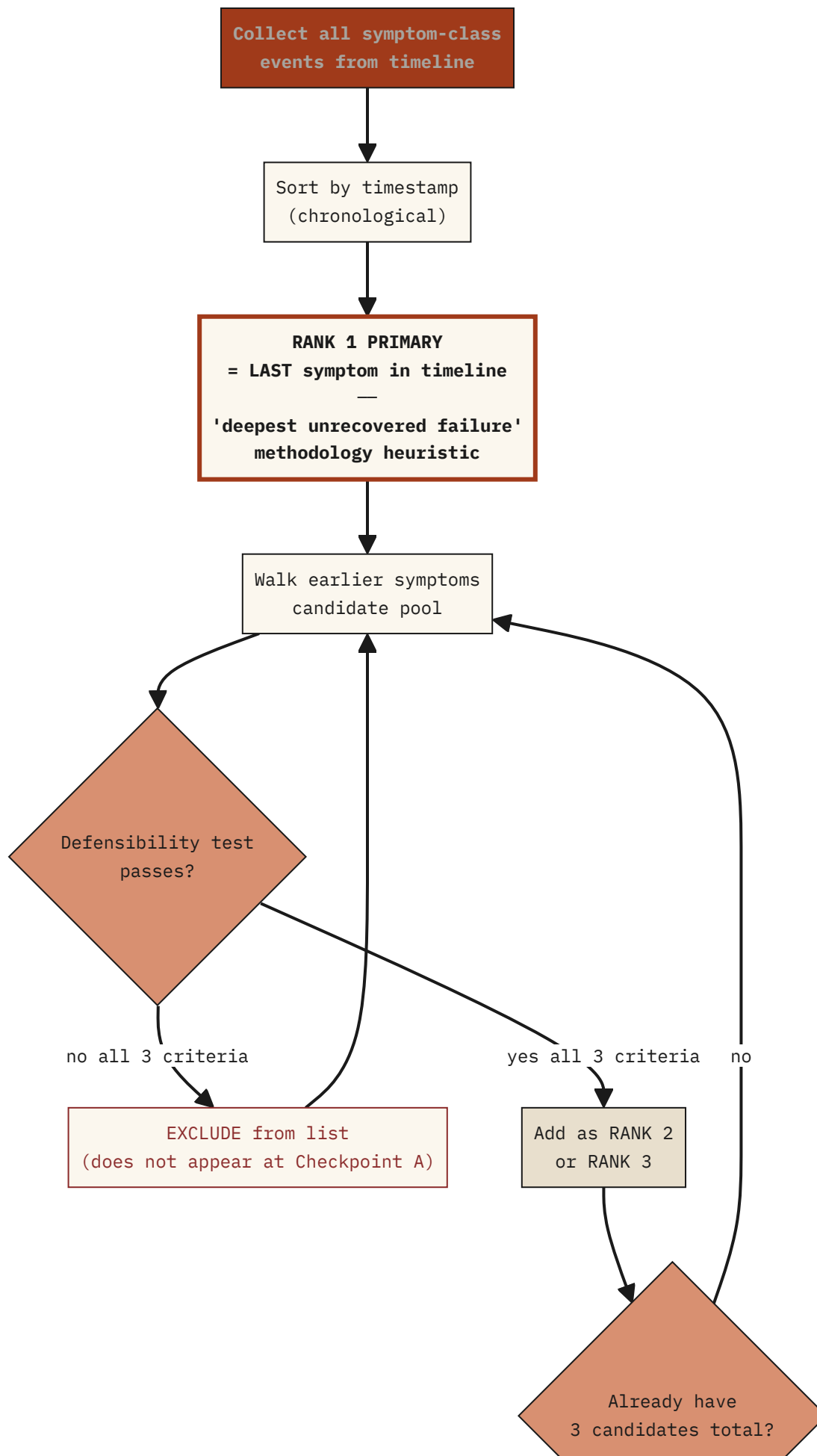
§07 Phase 2 · Building top_event_candidates[]

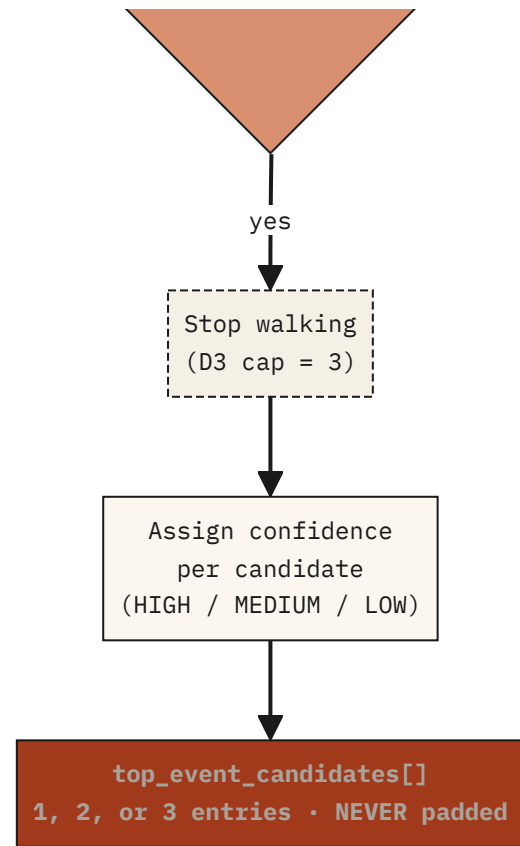
The Step 6 ranking algorithm is worth its own section because it is the most consequential output of Phase 2 – the curated list of candidates the user will choose from at Checkpoint A. Getting this list wrong (missing the right candidate, or padding it with weak ones) directly causes wasted FTA effort or rubber-stamped wrong root causes.

THE RANKING ALGORITHM

FIG. 4 – TOP_EVENT_CANDIDATES RANKING DECISIONS

FIG-04





THE DEFENSIBILITY TEST (3 CRITERIA · ALL REQUIRED)

A symptom can become an alternative candidate (rank 2 or 3) only if it satisfies **all three** of these criteria. The conjunction is deliberately strict to prevent padding the list with weak alternatives.

CRITERION	DEFINITION	WHY IT MATTERS
Distinct outcome	The symptom represents a distinct procedure outcome, not just a consequence of an earlier event in the same chain	Otherwise the alternative is just an upstream view of the same failure – adds noise without giving the user a real choice
Own evidence	The symptom has its own signaling evidence row (REJECT, timer-expiry, etc.) – not inferred from absence alone	If the alternative is purely inferred, the user can't see what makes it defensible – it looks like guessing
MEDIUM-distinct from primary	Differs from the primary on at least one of: protocol layer, cause code family, or procedure phase	Two candidates that differ only in timestamp are not really alternatives – they're the same failure twice

THE "NO PADDING" RULE (D3 = 3 CAP, NOT FLOOR)

D3 SETS A CEILING, NOT A TARGET

The locked decision in v6 says "Top Event alternatives ≤ 3 ". This is a maximum, not a minimum. The candidates list may have:

- **1 entry** – common for clean captures with one clear symptom. Checkpoint A presents it as "PRIMARY (no alternatives meet the bar). Proceed?"
- **2 entries** – primary plus one defensible alternative
- **3 entries** – primary plus two defensible alternatives

NEVER 3 entries when only 1 or 2 are defensible. Inventing alternatives to fill out the menu defeats the purpose of curating it. A single defensible candidate presented for confirmation is more honest – and more useful to the user – than three with two of them weak.

REJECTION REASONS FOR NON-PRIMARY CANDIDATES

Each non-primary entry in the list has a `rejection_reason` field – a sentence explaining why the agent did not pick this one as the primary. This is the engineer's information for the Checkpoint A decision. Typical rejection reasons:

- "Consequence of primary; not a distinct top event" – failed the distinct-outcome criterion
- "Earlier in chain but recovers; primary is the unrecovered failure" – the alternative happened first but the procedure ultimately failed at the primary
- "Different layer but lower confidence; primary has stronger evidence" – both are real symptoms but primary has more support
- "Same procedure phase as primary; not meaningfully distinct" – failed the MEDIUM-distinct criterion

The `rejection_reason` for the primary candidate is always `null` .

WHAT IF NO SYMPTOM EVENT IS FOUND AT ALL?

THE "NO SYMPTOM" EDGE CASE

If Phase 2 walks the entire retrieved timeline and finds no symptom-class events – only normal, anomaly, and consequence events – there are no candidates to present. The skill then constructs an inferred candidate from the strongest anomaly evidence (e.g. "Procedure incomplete – RRCReconfigurationComplete absent within expected window") with LOW confidence and adds a note to ambiguities.

This is not the same as padding. The single inferred candidate IS the strongest defensible thing the agent can present, and Checkpoint A's "refine" or "reject and restart" options give the user a clean path forward. The alternative – Phase 2 silently producing nothing – would force a halt with no information for the engineer to act on.

§08 Phase 2 • Worked example

Continuing the 5G handover scenario. Phase 1 produced a clean `scope_filter` with no ambiguities; Phase 2 now consults the spec for the expected message flow and retrieves the actual signaling timeline.

STEP 2 – SPEC_QUERY EXPECTED FLOW (NEED=ECF)

```
spec_query.py --operation lightweight_procedure \
  --procedure "Intra-AMF 5G Handover" \
  --rat "5G NR" \
  --need ecf \
  --state-file ...

→ {
  "operation": "lightweight_procedure",
  "need": "ecf",
  "procedure": "Intra-AMF 5G Handover",
  "expected_flow": [
    { "order": 1, "message": "MeasurementReport", "direction": "UL", "layer": "RRC" },
    { "order": 2, "message": "RRCReconfiguration with reconfigurationWithSync",
      "direction": "DL", "layer": "RRC" },
    { "order": 3, "message": "Random Access Preamble", "direction": "UL", "layer": "MAC" },
    { "order": 4, "message": "Random Access Response (RAR)", "direction": "DL", "layer": "MAC" },
    { "order": 5, "message": "RRCReconfigurationComplete", "direction": "UL", "layer": "RRC" },
  ],
  "key_timers": [{ "timer": "T304", "duration_default": "~1000ms", "on_expiry": "HO fai" },
  "spec_refs": ["TS 38.331 §5.5.1.2"]
}
```

STEP 3 – LOG_QUERY SIGNALING TIMELINE

```
log_query.py --phase-tag phase2 --table UE_3gpp_signaling_log \
  --keywords "MeasurementReport" "RRCReconfiguration" \
    "Random Access" "RAR" "RRCReconfigurationComplete" \
  --time-window-start-ms 14020000 --time-window-end-ms 14023000 \
  --layers "RRC" "MAC" "PHY" \
  --state-file ...

→ retrieved 4 rows (of 5 expected); plus T304 expiry message
key_events: [
  { ts: "14:02:10.30", layer: "RRC", direction: "UL", message: "MeasurementReport" },
  { ts: "14:02:11.00", layer: "RRC", direction: "DL",
    message: "RRCReconfiguration",
    kpis: { reconfigurationWithSync: true } },
  { ts: "14:02:11.05", layer: "MAC/PHY", direction: "UL", message: "Random Access Pre
  { ts: "14:02:12.00", layer: "RRC", direction: "UL",
    message: "T304 expiry → RRC Connection Reestablishment Request" },
  { ts: "14:02:12.50", layer: "RRC", direction: "DL",
    message: "RRC Connection Reestablishment Reject", cause_code: "other" }
]
```

STEP 4 – EVENT CLASSIFICATION

TIMESTAMP	MESSAGE	CLASSIFICATION	REASONING
14:02:10.30	MeasurementReport	✓ normal	Matches expected_flow order 1
14:02:11.00	RRCReconfiguration	✓ normal	Matches expected_flow order 2; correct DL
14:02:11.05	Random Access Preamble	✓ normal	Matches expected_flow order 3
14:02:12.00	T304 expiry	⚠ anomaly	First deviation – T304 expired without RAR or completion. earliest_anomaly

TIMESTAMP	MESSAGE	CLASSIFICATION	REASONING
14:02:12.50	RRC Reestab Reject	✗ symptom	Explicit failure outcome – reject with cause_code

earliest_anomaly = "14:02:12.00 – T304 expiry"

STEP 5 – MISSING EVENTS

```
missing_events = [  
  { "expected_message": "Random Access Response (RAR)",  
    "expected_after": "Random Access Preamble",  
    "spec_ref": "TS 38.321 §5.1" },  
  { "expected_message": "RRCReconfigurationComplete",  
    "expected_after": "Random Access Response (RAR)",  
    "spec_ref": "TS 38.331 §5.5.1.2" }  
]
```

STEP 6 – BUILD TOP_EVENT_CANDIDATES[]

Symptoms in chronological order:

1. 14:02:12.00 – T304 expiry (anomaly, but in the sequence a T-timer expiry is also a procedure outcome → could be a symptom)
2. 14:02:12.50 – RRC Connection Reestablishment Reject (definitive symptom)

The agent reconsiders: T304 expiry IS the moment the procedure formally failed. The reestablishment reject is the consequence of the HO failure. Both are symptom-class but they describe different things:

- The first describes the original handover's outcome: **5G_HO_Execution_Failure** (no target cell sync after RRCReconfiguration)
- The second describes the recovery attempt's outcome: RRC reestablishment reject

By the methodology heuristic "primary = last unrecovered symptom", the reestablishment reject is later in time but is a consequence; the HO failure is the unrecovered original event. **Primary = HO execution failure.**

Now apply the defensibility test to the reestablishment reject as a potential alternative:

- **Distinct outcome?** Yes – it's a different procedure (reestablishment, not handover)
- **Own evidence?** Yes – REJECT message with cause_code
- **MEDIUM-distinct from primary?** Marginal – same RRC layer, different procedure phase. Pass.

The reestablishment reject passes the defensibility test → goes in as rank 2 with rejection_reason "Consequence of primary; reestablishment after HO failure".

STEP 7 – LOG COVERAGE

Expected_flow had 5 entries; 4 distinct expected messages were retrieved (Measurement Report, RRCReconfiguration, RA Preamble, plus the T304 expiry which is procedure-related though not in expected_flow). Missing: RAR, RRCReconfigurationComplete.

$\text{coverage} = 3/5 = 0.6 \rightarrow \text{partial}$

STEP 8 – CONFIDENCE ASSIGNMENT

- **Rank 1 (HO execution failure):** coverage=partial caps at MEDIUM, BUT the symptom has very strong inferred-from-absence evidence (RRCReconfiguration with sync received, no RAR within T304 window – exactly the textbook signature). The skill upgrades to **HIGH** via the "strong inferred-from-absence" rule for MEDIUM.
- **Rank 2 (Reestab reject):** coverage=partial, evidence is direct REJECT row → **LOW** (because it's a consequence, the strength of evidence doesn't elevate it past the primary)

STEP 9 – AMBIGUITY CHECK

coverage=partial AND rank 1 is HIGH confidence → no ambiguity note needed. Proceed.

STEP 10 – FINAL PHASE2_ECF WRITTEN

```

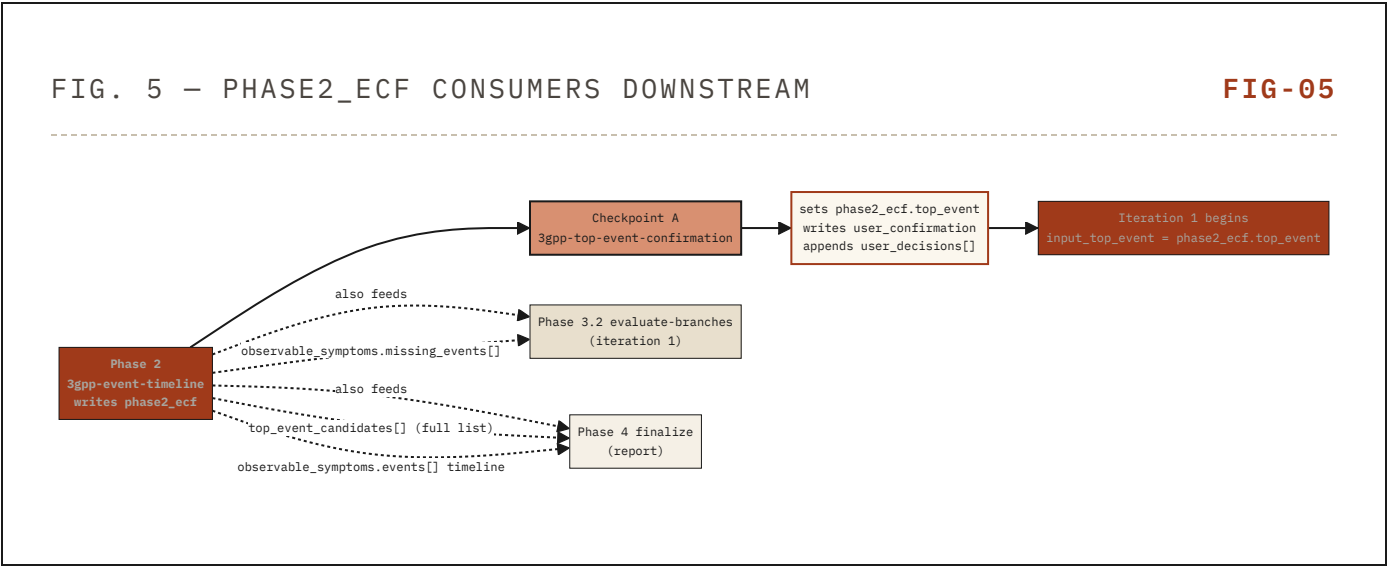
phase2_ecf = {
  completed_at: "2026-05-20T03:19:55Z",
  top_event: null,                                     // ← Checkpoint A will set
  observable_symptoms: {
    events: [/* 5 classified events */],
    earliest_anomaly: "14:02:12.00 - T304 expiry",
    missing_events: [
      { expected_message: "Random Access Response (RAR)",
        expected_after: "Random Access Preamble",
        spec_ref: "TS 38.321 §5.1" },
      { expected_message: "RRCReconfigurationComplete",
        expected_after: "Random Access Response (RAR)",
        spec_ref: "TS 38.331 §5.5.1.2" }
    ],
    log_coverage: "partial",
    queried_tables: ["UE_3gpp_signaling_log"],
    expected_flow_source: { /* full spec_query output */ }
  },
  top_event_candidates: [
    {
      rank: 1, is_primary: true,
      event: "5G_H0_Execution_Failure (no target cell sync)",
      timestamp: "14:02:11.05 onwards (T304 expiry at 14:02:12.00)",
      layer: "RRC/PHY",
      evidence: "RRCReconfiguration with reconfigurationWithSync received at 14:02:11.05"
        + " + RA Preamble sent at 14:02:11.05; no RAR observed; T304 expired at 14:02:12.00",
      confidence: "HIGH",
      rejection_reason: null
    },
    {
      rank: 2, is_primary: false,
      event: "RRC Connection Reestablishment Reject",
      timestamp: "14:02:12.50",
      layer: "RRC",
      evidence: "Reestablishment Reject with cause_code 'other' after T304 expiry",
      confidence: "LOW",
      rejection_reason: "Consequence of primary; reestablishment attempted only after H
    }
  ],
  user_confirmation: null                               // ← Checkpoint A will set
}

```

Phase 2 complete. `meta.current_phase` = "phase2_running" (unchanged); the workflow next dispatches `3gpp-top-event-confirmation` which transitions phase to `phase2_pending_confirmation` and presents Checkpoint A.

§09 Handoff to Checkpoint A & Phase 3

Phase 2's output is consumed at two distinct downstream stages. Understanding which field flows to which stage clarifies why Phase 2 deliberately leaves some fields null.



WHICH PHASE2_ECF FIELD IS CONSUMED WHERE

FIELD	SET BY	CONSUMED BY
phase2_ecf.top_event_candidates[]	Phase 2 (this skill)	Checkpoint A – rendered as the candidate list the user picks from. Also rendered in Phase 4 report Section 2 as the "alternatives considered table."
phase2_ecf.observable_symptoms.events[]	Phase 2 (this skill)	Phase 4 report Section 2 – the signaling timeline rendered as evidence content
phase2_ecf.observable_symptoms.missing_events[]	Phase 2 (this skill)	Phase 3.2 evaluate-branches iteration 1 – branches whose mandatory_messages match

FIELD	SET BY	CONSUMED BY
		missing_event get evaluate first (pivot-prune priority).
phase2_ecf.observable_symptoms.log_coverage	Phase 2 (this skill)	Checkpoint A – included in the prompt header so the user sees coverage state when choosing.
phase2_ecf.top_event	Checkpoint A skill (not Phase 2)	Iteration 1's input_top_event ; consumed by 3gpp-fta-bui tree as the root of the iteration 1 fault tree.
phase2_ecf.user_confirmation	Checkpoint A skill (not Phase 2)	Phase 4 report Section 6 (user decision audit); also tracks overrode_recommendati for the disagreement flag.

THE USER'S OPTIONS AT CHECKPOINT A (WHAT PHASE 2 ENABLES)

Phase 2's output determines what the user can do at Checkpoint A:

PHASE 2 OUTPUT STATE	CHECKPOINT A BEHAVIOR
SINGLE HIGH CANDIDATE	Confidence is high enough that "confirm" is the obvious choice. The user can still refine or reject, but the path of least resistance is acceptance.
PRIMARY + 1-2 DEFENSIBLE ALTERNATIVES	The user sees the full menu with evidence and rejection_reasons. "use alternative 2" or "use alternative 3" are first-class options.
CANDIDATE(S) WITH LOW CONFIDENCE	The agent's recommendation is still "confirm rank 1" but with a confidence note. The user is more likely to choose "refine" to add context, or "reject and restart" to recapture.

PHASE 2 OUTPUT STATE	CHECKPOINT A BEHAVIOR
NO SYMPTOM FOUND · INFERRED CANDIDATE	Coverage is so poor that the candidate is essentially a placeholder. User should typically choose "refine" or "reject and restart". Checkpoint A's "abort" option is available if the engineer wants to escalate.

THE REFINE PATH (RE-ENTRY INTO PHASE 2)

If the user types `/rca refine: <text>` at Checkpoint A, the Checkpoint A skill:

1. Appends the refinement text to `phase1_scope_filter.user_refinements[]` (an array – multiple refinements accumulate)
2. Clears `phase2_ecf` entirely
3. Sets `meta.current_phase = "phase2_running"`
4. HALTs – user types `/rca` again to trigger the next dispatch

The next dispatch re-runs Phase 2 (this skill) with the refined context picked up in Step 1. Phase 1's `scope_filter` is otherwise unchanged – only the refinement context shifts what Phase 2 retrieves and how it ranks candidates.

WHY REFINE DOESN'T RESTART PHASE 1

A refine typically narrows or adds context within the same scope – "look only at 14:02:11-14:02:12 not the whole window" or "include the MAC layer in the timeline". The original `scope_filter` is still valid. If the scope itself is wrong, the user picks "reject and restart" which DOES restart Phase 1.

§10 Quick reference · halts & lookups

HALT CONDITIONS ACROSS PHASE 1 + PHASE 2

CONDITION	PHASE	TRIGGER	RESUME ACTION
Procedure unrecognized	1	spec_query lightweight_procedure returned empty phases/spec_refs	Engineer confirms procedure name; /rca resumes Phase 1
Ambiguities present	1	ambiguities[] non-empty at Step 5	Engineer clarifies each item via prose; pipeline appends and re-runs Phase 1
Table isolation violation	1 or 2	Skill attempts to query UE_Trace_log via log_query.py	Hard fault (script exit 3); fix skill; restart from current phase
expected_flow empty	2	spec_query lightweight_procedure (need=ecf) returned empty	Rare – engineer confirms procedure name (should have been caught Phase 1)
Sparse coverage + no HIGH candidate	2	log_coverage=sparse AND all candidates LOW	NOT a halt – note added to ambiguities; Checkpoint A handles via refine/reject options
No symptom found	2	Walked timeline; no symptom-class events identified	Inferred LOW-confidence candidate produced; Checkpoint A surfaces situation

EVENT CLASSIFICATION – DECISION CRITERIA

CLASS	TRIGGERS (ANY MATCHES)	EFFECT ON CANDIDATES
NORMAL	Matches expected_flow at expected position; no failure indicators; correct direction; cause_code absent or 0	Timeline context only; not a candidate
ANOMALY	First deviation from expected_flow – wrong message, wrong order, timer expiry, or unexpected cause code at any position	Sets earliest_anomaly; may or may not be a candidate (depends on Step 6 ranking)
CONSEQUENCE	Appears after an earlier anomaly AND is plausibly derivative (reestablishment, retry, recovery attempt)	Usually rejected as candidate (rejection_reason: "consequence of primary")
SYMPTOM	REJECT message, T-timer expiry, NoService entry, explicit failure indicator at application level	Candidate pool – primary is the LAST symptom; alternatives must pass defensibility test

CONFIDENCE ASSIGNMENT – DECISION CRITERIA

CONFIDENCE	LOG_COVERAGE REQUIREMENT	EVIDENCE REQUIREMENT
HIGH	rich (≥ 0.8)	Explicit signaling evidence – REJECT with cause code, T-timer expiry message, or unambiguous symptom row. Or: partial coverage + textbook inferred-from-absence signature.
MEDIUM	partial (0.4–0.79) OR rich with indirect evidence	Inferred from message absence with strong context; or direct evidence in partial-coverage capture.
LOW	sparse (< 0.4) OR partial without strong inference	Indirect signals only; or sparse capture even with direct evidence.

DEFENSIBILITY TEST (ALTERNATIVE CANDIDATES · ALL 3 REQUIRED)

CRITERION	PASS CONDITION
Distinct outcome	The symptom represents a different procedure outcome than the primary, not just an upstream/downstream view of the same failure
Own evidence	The symptom has its own direct signaling evidence row (REJECT, timer-expiry, etc.); not purely inferred from absence
MEDIUM-distinct from primary	Differs from primary on at least one of: protocol layer, cause code family, procedure phase

STATE FILE WRITE SUMMARY – PHASE 1 + PHASE 2

STATE FILE PATH	WRITTEN BY	WHEN
phase1_scope_filter	3gpp-scoping	Phase 1 Step 6
phase1_scope_filter.spec_lookup	spec_query.py	Phase 1 Step 3 (auto)
phase1_scope_filter.signaling_sanity_check	log_query.py	Phase 1 Step 4 if inv (auto-write)
phase1_scope_filter.user_refinements[]	3gpp-top-event-confirmation	Checkpoint A "refin
phase2_ecf.observable_symptoms	3gpp-event-timeline	Phase 2 Step 10
phase2_ecf.observable_symptoms.expected_flow_source	spec_query.py	Phase 2 Step 2 (auto)
phase2_ecf.top_event_candidates[]	3gpp-event-timeline	Phase 2 Step 10
phase2_ecf.top_event	3gpp-top-event-confirmation	After Checkpoint A response
phase2_ecf.user_confirmation	3gpp-top-event-	After Checkpoint A response

STATE FILE PATH	WRITTEN BY	WHEN
confirmation		
user_decisions[] (Checkpoint A entry)	3gpp-top-event-confirmation	After Checkpoint A response
meta.current_phase	each skill on exit	phase1 → phase2_r phase2_pending_co → phase2_confirme
keyword_provenance_audit[]	spec_query.py + log_query.py (auto-append)	Every tool invocatio Phase 1 + Phase 2

TOOL INVOCATION SUMMARY – PHASE 1 + PHASE 2

TOOL · OPERATION	PHASE	REQUIRED?	PURPOSE
spec_query lightweight_procedure (need=is_is_not)	1	REQUIRED	Get primary_layers, key_timers, initiating_message, spec_refs
log_query phase1 (signaling)	1	OPTIONAL	Sanity-check that initiating_message appears in capture
spec_query lightweight_procedure (need=ecf)	2	REQUIRED	Get expected_flow[] for comparison and keyword list
log_query phase2 (signaling)	2	REQUIRED	Retrieve the actual signaling timeline for classification

Total: 2 required tool calls, 1 optional. Phase 1 + Phase 2 together cost roughly 1.5K-3K tokens of tool I/O – cheap compared to the FTA phase. The cost discipline matters because the workflow may re-run Phase 2 multiple times if the engineer refines at Checkpoint A.

Phase 1 + Phase 2 – Detailed Procedure

v6 · Scope & Top Event pipeline

Companion to [fta-detailed-procedure.html](#)Worked example: 5G H0 failure · [pre-FTA prologue](#)